

NeuralODE for Planar Pushing

Hande Huang

Abstract—We investigate the use of neural ordinary differential equations (Neural ODEs [1]) to model and predict the quasi-static dynamics of a planar pushing task. Building on classical residual and absolute dynamics networks, we directly learn the continuous-time vector field governing object motion under robot pushes. We systematically explore how integration algorithm (Euler, RK4, Dormand–Prince), prediction horizon, learning rate, and network expressivity affect single- and multi-step predictive performance. Our best Neural ODE matches or outperforms a comparable discrete-time residual model, achieving sub- 10^{-4} validation losses while requiring fewer hyperparameter sweeps. Finally, we embed the learned continuous dynamics in a model-predictive path-integral (MPPI) controller to plan obstacle-free and obstacle-avoiding pushes, demonstrating reliable goal achievement in under 20 simulated steps.

I. INTRODUCTION

Accurately modeling object motion under contact is fundamental to advanced robotic manipulation. In many real-world settings—warehouse automation, assembly lines, or household assistance—a robot must push, slide, or reorient objects reliably, often in cluttered environments. Classical physics-based models of frictional contact are computationally expensive and sensitive to unmodeled uncertainties (surface variations, unobserved forces), while purely data-driven discrete-time neural networks can struggle with long-horizon stability and require retraining when dynamics subtly change.

Neural ODEs offer a powerful alternative by learning a continuous-time vector field directly from state-action trajectories. This formulation naturally handles variable time steps, supports adaptive integrators for efficiency, and imposes physical smoothness via the ODE structure. When paired with model-predictive control (e.g., MPPI), a Neural ODE dynamics model can be rolled out recursively to plan multi-step push sequences—both in open space and around obstacles—while respecting learned frictional and geometric constraints.

In this work, we apply Neural ODEs to the canonical planar pushing problem, where a 7-DoF robot contacts a rectangular block on a table. We first collect randomized push trajectories, then train both single-step and multi-step Neural ODE models across various integrators and network sizes. We benchmark against residual neural networks from prior work, showing that properly tuned Neural ODEs achieve equal or superior predictive accuracy. Finally, we integrate the learned dynamics into an MPPI controller to solve free-space and obstacle-avoidance pushing tasks, illustrating the method’s potential for robust, data-driven manipulation in complex environments.

II. IMPLEMENTATION

A. Neural ODE Formulation

The core implementation of this project is the formulation of Neural ODE. Instead of using the discrete-time residual update

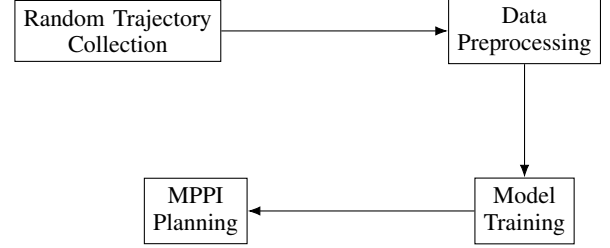


Fig. 1. Block diagram of the overall implementation pipeline.

from HW3, we directly learn a continuous-time dynamics of the planar pushing system as an ordinary differential equation on the augmented state-action vector $[x; u]$:

$$\frac{d}{dt} \begin{bmatrix} x(t) \\ u(t) \end{bmatrix} = \underbrace{\begin{bmatrix} f_\theta(x(t), u(t)) \\ 0 \end{bmatrix}}_{g_\theta(x(t), u(t))}, \quad x(0) = x_t, \quad u(t) \equiv u_t. \quad (1)$$

Here, $x \in \mathbb{R}^3$ is the block pose in $SE(2)$ and $u \in \mathbb{R}^3$ is the push parameter. We approximate f_θ using a multilayer perceptron (MLP) with hidden sizes $\{h_i\}$.

```

class ODEFunc(nn.Module):
    def __init__(self, state_dim, action_dim,
                 hidden_sizes):
        super().__init__()
        self.state_dim = state_dim
        layers = []
        inp_dim = state_dim + action_dim
        for h in hidden_sizes:
            layers += [nn.Linear(inp_dim, h),
                      nn.ReLU()]
            inp_dim = h
        layers += [nn.Linear(inp_dim,
                             state_dim)]
        self.net = nn.Sequential(*layers)

    def forward(self, t, xa):
        dx = self.net(xa)
        du = torch.zeros_like(xa[... ,
                                self.state_dim:])
        return torch.cat([dx, du], dim=-1)
  
```

To predict the next discrete state x_{t+1} from (x_t, u_t) , we integrate (1) over one time-step:

$$\hat{x}_{t+1} = x_t + \int_0^{\Delta t} f_\theta(x(\tau), u_t) d\tau. \quad (2)$$

In practice, we set $\Delta t = 1$ and integrate using `torchdiffeq.odeint_adjoint[2]`:

```

class NeuralODEModel(nn.Module):
    def __init__(self, state_dim, action_dim,
                 hidden_sizes, method='rk4'):
  
```

```

    super().__init__()
    self.func = ODEFunc(state_dim,
        action_dim, hidden_sizes)
    self.method = method

    def forward(self, state, action, dt=1.0):
        xa0 = torch.cat([state, action],
            dim=-1)
        t = torch.tensor([0., dt],
            device=state.device)
        # Use adjoint method for memory
        # efficiency
        xa_t = odeint_adjoint(self.func, xa0,
            t, method=self.method)
        return xa_t[-1][..., :state.shape[-1]]

```

B. The Rest

All other components in the pipeline are inherited from the HW3 framework, with only minor modifications. We briefly summarize them here:

a) Data Collection and Processing:

- `collect_data_random(env, ...)` runs N random push trajectories of length T , returning a list of dictionaries $\{\{ \text{'states'} : [x_0, \dots, x_T], \text{'actions'} : [u_0, \dots, u_{T-1}] \}\}$.
- `SingleStepDynamicsDataset` and `MultiStepDynamicsDataset` wrap these trajectories into (x_t, u_t, x_{t+1}) pairs or $(x_t, \{u_{t:t+K-1}\}, \{x_{t+1:t+K}\})$ subsequences, respectively.
- We split each dataset 80/20 via `torch.utils.data.random_split` and load with `DataLoader`.

b) Loss Functions:

- **SE2PoseLoss**: for any predicted $\hat{x} = (x, y, \theta)$ and target x ,

$$\mathcal{L}_{\text{pose}} = \|\hat{x}_{1:2} - x_{1:2}\|_2^2 + r_g (\hat{\theta} - \theta)^2, \quad r_g = \sqrt{\frac{w^2 + \ell^2}{12}}.$$

- **SingleStepLoss** wraps $\mathcal{L}_{\text{pose}}$ to compare $\hat{x}_{t+1} = \text{model}(x_t, u_t)$ vs. the true x_{t+1} .
- **MultiStepLoss** rolls the model forward K steps and accumulates discounted step losses $\sum_{k=0}^{K-1} \gamma^k \mathcal{L}_{\text{pose}}(\hat{x}_{t+k+1}, x_{t+k+1})$.

c) *Residual Dynamics Baseline*: As in HW3, we also implement the discrete-time residual model

$$x_{t+1} = x_t + f_\phi(x_t, u_t),$$

via the `ResidualDynamicsModel` MLP. This serves as a direct baseline for comparison against our Neural ODE.

d) Cost Functions and Controller:

- `free_pushing_cost_function` computes a quadratic state cost $(x - x_{\text{goal}})^\top Q (x - x_{\text{goal}})$.
- `collision_detection` implements a separating-axis check between the block and a rectangular obstacle.
- `obstacle_avoidance_pushing_cost_function` adds a large penalty (100) whenever `collision_detection(x) = 1`.

- `PushingController` wraps the learned model and cost into an MPPI optimizer, with user-tuned noise covariance, temperature, horizon, and sample count.

Together, these modules provide data I/O, baselines, losses, and planning logic, completing the end-to-end implementation inherited from HW3.

III. RESULTS

Note: for GIFs of the planar pushing simulations, please see the demo.

A. Experimental Setup

All experiments were carried out in a simulated planar-pushing environment built on PyBullet and wrapped as a Gym interface (`PandaPushingEnv`). We ran everything on a machine with an NVIDIA GPU (when available) under Python 3.11.5, PyTorch 2.5.1, CUDA 12.1, and `torchdiffeq` for the ODE integration.

Data Collection. We gathered $N = 100$ random push trajectories of length $T = 10$ steps. At each step the action u_t was sampled uniformly via `env.action_space.sample()`, and the resulting next pose x_{t+1} was recorded (then cast to `float32`). Trajectories were saved as a NumPy `.npy` file for later reuse.

Data Processing. Two custom PyTorch datasets were implemented in `learning_state_dynamics.py`:

- `SingleStepDynamicsDataset`: all (x_t, u_t, x_{t+1}) pairs (total $100 \times 10 = 1000$ samples),
- `MultiStepDynamicsDataset` with $K = 4$ steps (total $100 \times (10 - 4 + 1) = 700$ samples).

Each was split 80%/20% into training/validation via `torch.utils.data.random_split`, and loaded in mini-batches of size 500.

Neural ODE Testing Details. We install and import the adjoint ODE solver from `torchdiffeq`:

```

from torchdiffeq import odeint_adjoint as
odeint

```

Our dynamics model is wrapped in

```

NeuralODEModel(
    state_dim=3,
    action_dim=3,
    hidden_sizes=[100, 100],
    method=<integrator>
)

```

where `method` is one of `{euler, rk4, dopri5}`. In its forward pass, we concatenate the state-action vector $[x; u]$, integrate

$$\dot{x} = f_\theta(x, u), \quad \dot{u} = 0$$

over $[0, \Delta t]$ (with $\Delta t = 1$) via `odeint_adjoint`, and return the new state.

For single-step fitting we define the pose loss

```

pose_loss = SE2PoseLoss(block_width=0.1,
    block_length=0.1)
step_loss = SingleStepLoss(pose_loss)

```

and for multi-step fitting we use

```
step_loss = MultiStepLoss(pose_loss,
    discount=0.9)
```

on 4-step sequences assembled by `process_data_multiple_step`.

All training happens on GPU when available. We move both the model and each batch (`state`, `action`, `next_state`) to device, zero the Adam optimizer, backpropagate `step_loss(model, ...)`, and step the optimizer. For both single and multiple-step models, we swept hyperparameters (learning rate, integration algorithm, network expressivity) in with multiple stages of ranges (see next section), and logged training and validation losses after every 20 epochs (shown on a log scale). The following section will be dedicated to these procedures.

MPPI Controller. We re-used the HW2 MPPI implementation (`mppi.py`), configuring **MPPI Controller**. We reuse our HW2/3 MPPI (`mppi.py`), with horizon $H \in \{10, 20\}$, $\text{num_samples} \in \{100, 1000\}$, $\Sigma = 0.4I$, $\lambda = 0.01$, and $u_{\min}, u_{\max}$ taken from the env. We tested both obstacle-free pushing (quadratic cost with $Q = \text{diag}(1, 1, 0.1)$) and obstacle avoidance (adding $100\times$ the binary collision cost).

B. Results in sequence

1) *Grid Search of the Single-step NeuralODE Model Hyperparameters:* We first do a broad grid search over learning rates $[1e-2, 1e-3, 1e-4]$ and integration methods $['\text{euler}', '\text{rk4}', '\text{dopri5}']$ over 150 epochs each to get a sense of the scales.

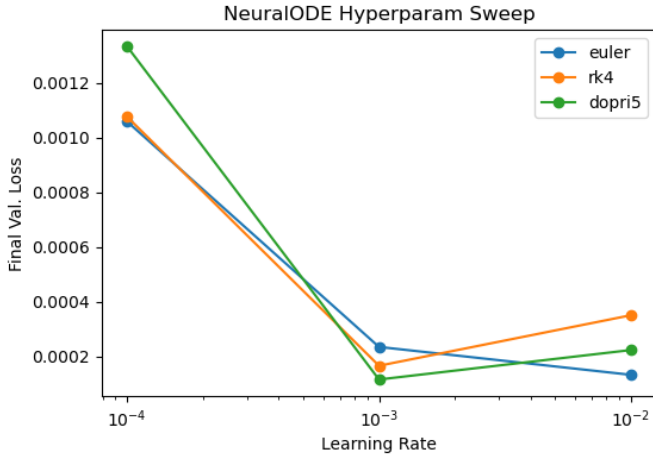


Fig. 2. A broad hyperparameter search for the single-step NeuralODE model

As we see in Figure2, the best learning rate is likely between $1e-2$ and $1e-3$, with rk4 and dopri5 favoring near $1e-3$. We can also see that for the euler method, it takes a while to converge (still not converging at the end of the $1e-2$ learning rate run), while at that time, rk4 and dopri5 have already overfitted. This is consistent with what we know about these integration methods - in general, $\text{dopri5} > \text{rk4} > \text{euler}$ (in short, dopri5 is a more sophisticated version of rk5, and rk4 is a more sophisticated version of euler). We can also observe the dopri5

>rk4 part, note that dopri5 have lower validation losses than rk4 with the $1e-3$ and $1e-2$ cases, where the models converge between these two levels.

We then further narrow down the learning rate range to $[7e-4, 9e-4, 1e-3, 3e-3, 5e-3, 7e-3, 1e-2]$ with 1000 epoch, but this time with rk4 only due to time constraints, as dopri5 takes considerably longer time to train. We also implement the ReduceLROnPlateau strategy with patience = 4, factor = 0.8 to search for the best model.

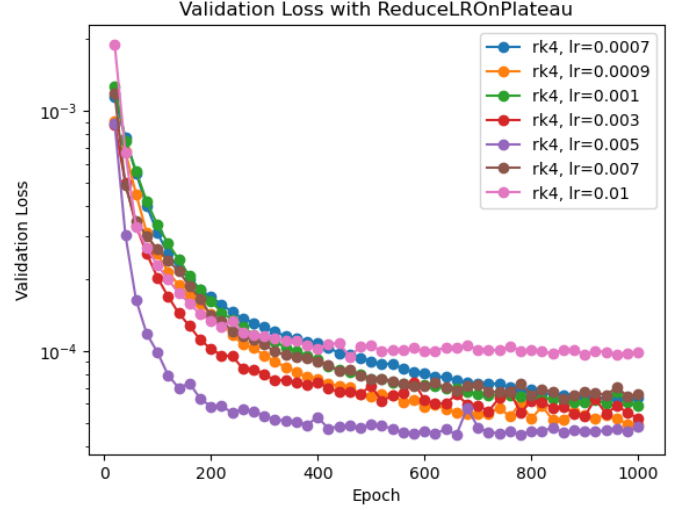


Fig. 3. A more sophisticated learning rate search on the single step model

As we see in Figure3, a learning rate of $5e-3$ (purple) seems like a good learning rate, with a final validation loss of $4.8048e-05$; although I must note that this "best starting learning rate" is not always consistent. We then test the best hyperparameters above on the dopri5 integration method.

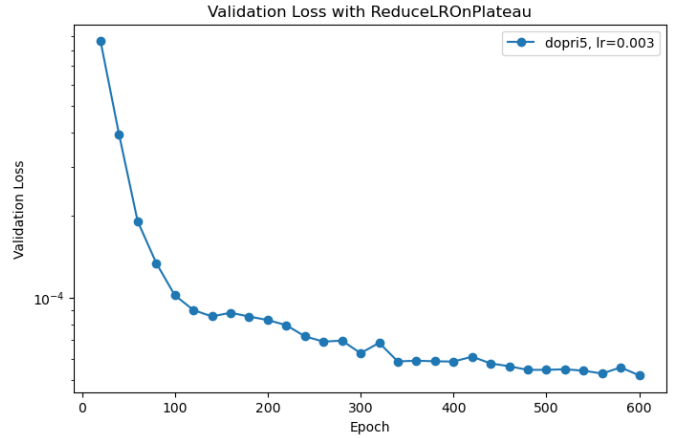


Fig. 4. Testing best rk4 hyperparameters on dopri5

In Figure4, the training ended with a loss of $5.2073e-05$ at 600 epochs, with a small room of improvement with further training. However due to time constraints, I did not re-train it with more epochs, as it is unlikely to improve much compared to the rk4's best validation loss of $4.8e-5$.

The conclusion here is that for single-step models, euler is outperformed by rk4 and dopri5, where the latter two are

roughly the same, with dopri5 likely to be slightly better, from what we know about the two integration methods (discussed earlier).

2) *Grid Search of the Multi-step NeuralODE Model Hyperparameters:* Similar to what we did for single-step models, we perform a grid search for multi-step models as well. Here, we start with only rk4, as we know euler is going to be outcompete, and dopri5 is taking forever to train. In the end we will test the best rk4 hyperparameters on dopri5 for a comparison between the integration methods. We first do a broad grid search with number of steps = [1, 4, 8], learning rates [1e-2, 1e-3, 1e-4], and 500 epochs.

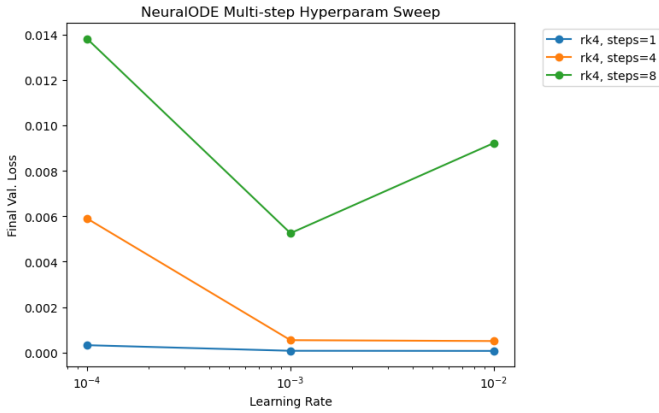


Fig. 5. Broad grid search on multi-step models with varying # of steps and learning rates

We can see in Figure5 that, clearly, smaller number of steps performs better, at least with 600 epochs; and the best learning rate is again around 1e-3. From the result above, we then focus on number of steps = [1, 4], separately, with learning rates = [7e-4, 1e-3, 3e-3, 5e-3, 7e-3] on a ReduceLRonPlateau strategy with factor=0.1 and patience=4, and lastly, 1000 epochs.

For number of steps = 1, we can see the results in Figure 6, where we found $lr = 0.001$ to be a good learning rate, with a final validation loss of 4.5713e-05.

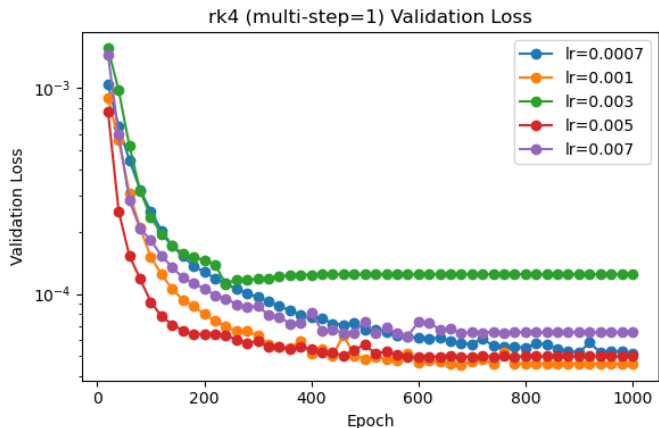


Fig. 6. Multi-step learning rate search on steps = 1

For number of steps = 4, we can see the results in Figure 7, where we found $lr = 0.003$ to be a good learning rate, with a final validation loss of 2.0823e-04.

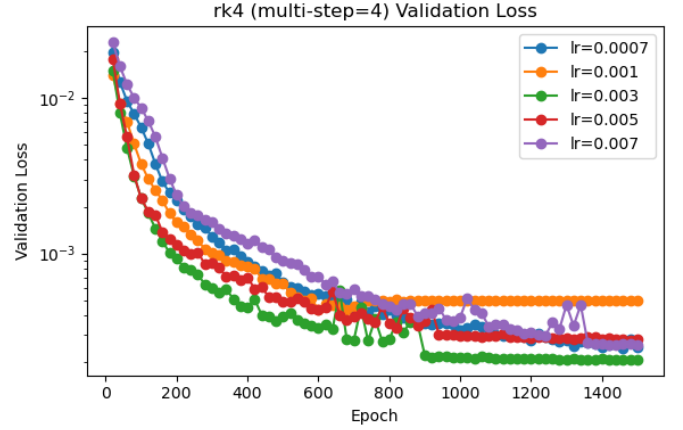


Fig. 7. Multi-step learning rate search on steps = 4

The conclusion here is that by examining the best losses from steps = 1 and steps = 4 models, we can confirm that smaller step multi-step models (steps = 1, in this case) are better for this specific task (planar pushing with this arm).

Finally, we take best hyperparameters for steps = 4 rk4 model and test them on the dopri5 model. I was not able to do a full learning rate grid search, so I only did a starting learning rate of 1e-2 with a ReduceLRonPlateau LR reduction strategy as before, on 1000 epochs. Even with this single test, I was able to achieve a validation loss of 5.0039e-4, as seen in Figure8 with room of improvement (not fully converged). Note that there is a spike at the end of the training, so I took the second last data point (loss = 5.0039e-4).

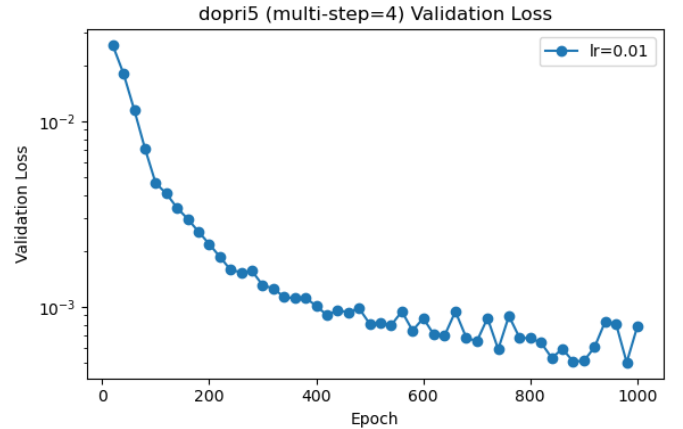


Fig. 8. Testing best rk4, # of steps = 4 hyperparameters on dopri5

The conclusion here is that by comparing the losses achieved between rk4 step4, 2.0823e-04, and the loss (without a sophisticated search) achieved by dopri5, 5.0039e-4, we can say that these two models performs roughly the same, with dopri5 having the potential to be better, with further parameter tuning and its inherent stronger integration method structure. However, the cost is that it would take much longer than rk4 to

find the best hyperparameters for dopri5 due to a much longer training time.

3) *Comparing Single-step and Multi-step NeuralODE Models*: In this section, we briefly compare single-step and multi-step models, like in Homework 3. On a new dataset, we compare the single-step model with the 4-step model. The results are shown below:

Validation single-step loss (single_model) : 7.008172×10^{-5}
Validation single-step loss (4-step_model) : 8.895261×10^{-5}
Validation multi-step loss (single_model) : 2.097441×10^{-4}
Validation multi-step loss (4-step_model) : 2.496874×10^{-4}

From above, we can see that single-step model outperforms multi-step model, with single-step loss outperforming multi-step loss. Note that these rankings are for this specific task (planar pushing with this arm) only.

4) *Network Expressivity Test of the NeuralODE Model*: In this section, we change hidden layer shapes and sizes. We set the integration method to rk4, learning rate to 3e-3, and train on 1000 epochs.

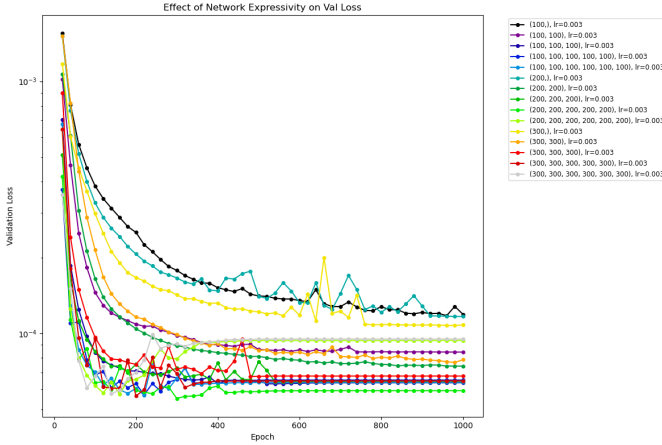


Fig. 9. Testing performance of different hidden layer sizes and shapes

Ranking results from Figure9 (loss ranked lowest to highest on the right column):

hidden=(200,200,200,200,200)	: 5.9252×10^{-5}
hidden=(100,100,100)	: 6.3657×10^{-5}
hidden=(100,100,100,100,100,100)	: 6.3883×10^{-5}
hidden=(200,200,200)	: 6.4544×10^{-5}
hidden=(300,300,300,300,300)	: 6.4791×10^{-5}
hidden=(100,100,100,100,100)	: 6.5254×10^{-5}
hidden=(100,200,100)	: 6.7413×10^{-5}
hidden=(300,300,300)	: 6.7698×10^{-5}
hidden=(200,200)	: 7.4118×10^{-5}
hidden=(50,100,50)	: 7.5918×10^{-5}
hidden=(300,300)	: 7.8561×10^{-5}
hidden=(100,100)	: 8.4248×10^{-5}
hidden=(200,200,200,200,200,200)	: 9.3678×10^{-5}
hidden=(300,300,300,300,300,300)	: 9.4916×10^{-5}
hidden=(300,)	: 1.0805×10^{-4}
hidden=(200,)	: 1.1665×10^{-4}
hidden=(100,)	: 1.1899×10^{-4}

The conclusion from the above is the following:

• **Shallow / under-parameterized nets under-fit:**

- Single hidden layer architectures [100], [200], [300] all yield the highest losses (approximately $1.1\text{--}1.2 \times 10^{-4}$).
- Narrow “bottleneck” architecture [50, 100, 50] also under-fits relative to deeper nets.

• **Moderate depth/width hits the sweet-spot:**

- Five layers of 200 units each ([200, 200, 200, 200, 200]) achieves the lowest loss $\approx 5.93 \times 10^{-5}$.
- Architectures [100, 100, 100] (6.37×10^{-5}), [100, 100, 100, 100, 100, 100] (6.39×10^{-5}), [200, 200, 200] (6.45×10^{-5}), and [300, 300, 300, 300, 300] (6.48×10^{-5}) cluster around $6.3\text{--}6.5 \times 10^{-5}$.
- These networks have sufficient capacity to capture the dynamics without unnecessary excess.

• **Over-parameterizing yields diminishing returns:**

- Extremely wide/deep nets such as [300, 300, 300, 300, 300, 300] and [200, 200, 200, 200, 200, 200] (6 layers) see losses rise back toward 9×10^{-5} .

5) *Loss Comparison: NeuralODE vs. Residual form Homework 3*: In this section, we take the Residual method from Homework 3 as a baseline, and compare it with the implemented NeuralODE method.

With a new dataset, comparing single-step and 4-step models, results are shown in Figure10.

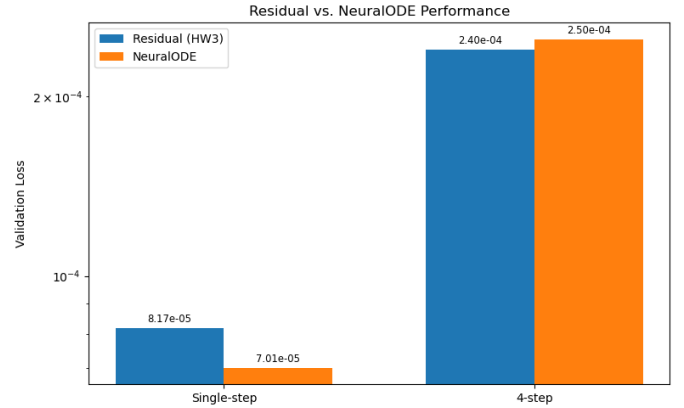


Fig. 10. Comparison between Residual and NeuralODE architectures

We can see that in the single-step case, NeuralODE method outperforms Residual method, and in 4-step, residual method outperforms NeuralODE method. However, the differences are small, and they aren't stable. I once achieved $3e-5$ validation loss for single-step NeuralODE method, compared to the $7e-5$ validation loss shown above. In the next section, we visually compare the two architectures by putting the models in simulation environment.

6) *Simulation Comparison*: Note: for GIFs of the simulation, please run demo.py.

We test 4 models, namely: single-step residual (from HW3), single-step NeuralODE, multi-step residual (from HW3), and multi-step NeuralODE on "obstacle free" and "with obstacle" cases, 5 times on each, and taking the average. For runs returning "false," in other words the model fails to complete the task, we count it as the maximum number of pushing actions allowed (20).

	Residual-single	NeuralODE-single	Residual-4step	NeuralODE-4step
Run 1	7	3	7	6
Run 2	8	3	6	6
Run 3	14	4	5	3
Run 4	6	5	6	3
Run 5	3	4	7	4
Average	7.6	3.8	6.2	4.4

TABLE I
OBSTACLE-FREE TEST

	Residual-single	NeuralODE-single	Residual-4step	NeuralODE-4step
Run 1	14	8	8	6
Run 2	False	11	18	6
Run 3	5	5	12	17
Run 4	False	5	12	7
Run 5	7	13	False	10
Average	13.2	8.4	14	9.2

TABLE II
WITH-OBSTACLE TEST

By examining the average steps in both obstacle-free (Table I) and with-obstacle (Table II) cases, we observe that the NeuralODE architecture outperforms the Residual architecture.

That concludes this project.

Thank you for the semester.

REFERENCES

- [1] R. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, "Neural ordinary differential equations," in Proceedings of the 32nd International Conference on Neural Information Processing Systems (NIPS), 2018.
- [2] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "torchdiffeq: Differentiable ODE Solvers for PyTorch," GitHub repository, 2018–2025. Available: <https://github.com/rtqichen/torchdiffeq>